

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE CODE: CSA1402

COURSE NAME: Compiler Design

COURSE OUTCOME COVERED

CO4: *Examine intermediate code generation techniques to enhance code optimization (BL4)*

Assessment Tool Weightage: 10%

QUESTIONS:

Total Marks:50

Annexure A

DATA INTERPRETATION

Code of Conduct:

I AL NAJEEB 192424325 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

SET 1

Scenario: Banking Transaction Processing System

A banking software company uses a domain-specific scripting language whose compiler translates variable declarations, assignment statements, and Boolean expressions into intermediate code. The system must also handle case statements for deposit, withdrawal, and loan classification.

Task (i) — Intermediate Code for Assignment and Boolean Expressions

The source-level banking expressions and their three-address code (TAC) intermediate representation are shown below. Three-address code is used because it is machine-independent, easy to optimise, and straightforward to translate to the target architecture.

Source code segment:

```
/* Variable declarations */
int balance, amount, limit;
bool isValid, isApproved;

/* Assignment statements */
balance = 50000;
amount  = 15000;
limit   = 40000;

/* Boolean expression for transaction validation */
isValid  = (amount > 0) AND (amount <= balance);
isApproved = (balance - amount) >= limit;

/* Assignment after validation */
if (isValid AND isApproved)
    balance = balance - amount;
```

Generated Three-Address Code (TAC):

```
(1) balance = 50000
(2) amount  = 15000
(3) limit   = 40000
(4) t1      = amount > 0
(5) t2      = amount <= balance
(6) isValid = t1 AND t2
(7) t3      = balance - amount
(8) t4      = t3 >= limit
(9) isApproved = t4
(10) t5     = isValid AND isApproved
```

```

(11) if t5 == false goto L1
(12) t6      = balance - amount
(13) balance = t6
(14) goto L2
(15) L1: /* transaction rejected */
(16) L2: /* continue */

```

Each complex expression is decomposed into simple binary operations with at most one operator per instruction. Temporary variables (t1–t6) hold intermediate results. This is the standard three-address code format used by most compiler back ends.

Task (ii) — Backpatching for Conditional Transaction Validation

Backpatching is the technique of deferring the filling-in of jump target addresses in conditional branch instructions until the target label becomes known during a single-pass code generation. In banking transaction validation, where multiple conditions must be checked before authorising a payment, backpatching is essential.

Consider the compound Boolean condition:

```

if (amount > 0 AND amount <= balance AND balance - amount >=
limit)
    processTransaction();
else
    rejectTransaction();

```

During single-pass generation, each sub-condition emits a conditional jump whose target is unknown at the time of emission:

```

(1)  t1 = amount > 0
(2)  if t1 == false goto ____    ;; truelist hole - backpatch
    later
(3)  t2 = amount <= balance
(4)  if t2 == false goto ____    ;; another hole
(5)  t3 = balance - amount
(6)  t4 = t3 >= limit
(7)  if t4 == false goto ____    ;; another hole
(8)  call processTransaction
(9)  goto L2
(10) L1: call rejectTransaction  ;; backpatch all holes to L1
(11) L2: ...

```

The compiler maintains a truelist and falselist for each Boolean expression. When the else label L1 is finally generated at step 10, all pending "goto ____" entries in the falselist are patched to point to L1. This avoids multiple passes over the intermediate code and is therefore both time-

efficient and memory-efficient, which is critical during peak banking hours when the compiler may process thousands of transaction scripts per second.

Task (iii) — Role of Intermediate Languages in Banking Software Portability

An intermediate language (IL) decouples the front end of the compiler, which understands the banking DSL syntax and semantics, from the back end, which generates machine code for a specific processor or virtual machine. This decoupling provides several portability advantages for banking software.

First, the same IL can be retargeted to multiple hardware platforms: x86-64 servers in the data centre, ARM processors in mobile banking terminals, and RISC-V cores in embedded point-of-sale devices. The front end is written once; only the back end changes per platform. Second, the IL acts as a platform-neutral contract: third-party auditors and regulators can inspect the IL representation of a transaction script without needing to understand the specific assembly language of each hardware platform. Third, IL enables platform-independent optimisations such as common subexpression elimination, dead code removal, and constant propagation. These optimisations are applied to the IL before code generation and therefore benefit every target platform equally. Fourth, when the banking company upgrades its server hardware, only the back end code generator needs to change, not the thousands of existing transaction scripts or the front-end parser. This dramatically reduces the cost and risk of hardware migrations, which are common in large financial institutions.

SET 2

Scenario: Industrial Robot Automation System

A manufacturing company's embedded compiler processes declarations, assignment statements, and Boolean expressions to coordinate robotic arm movement and safety interlocks. Delays caused by inefficient procedure call handling are to be resolved.

Task (i) — Intermediate Code for Robotic Movement and Safety Validation

Source-level control logic for a robotic arm:

```
/* Declarations */
int  motorSpeed, targetPos, currentPos;
bool safetyOK, positionReached;

/* Initial assignments */
targetPos  = 180;    /* degrees */
motorSpeed = 75;     /* RPM */

/* Safety check Boolean expression */
safetyOK = (currentPos >= 0) AND (currentPos <= 360) AND
(motorSpeed <= 100);

/* Movement conditional */
if (safetyOK)
    currentPos = currentPos + motorSpeed;

/* Check position reached */
positionReached = (currentPos == targetPos);
```

Three-Address Code (TAC):

```
(1)  targetPos  = 180
(2)  motorSpeed = 75
(3)  t1 = currentPos >= 0
(4)  t2 = currentPos <= 360
(5)  t3 = motorSpeed <= 100
(6)  t4 = t1 AND t2
(7)  safetyOK = t4 AND t3
(8)  if safetyOK == false goto L_HALT
(9)  t5 = currentPos + motorSpeed
(10) currentPos = t5
(11) goto L_CHECK
(12) L_HALT: /* emergency stop */
(13) L_CHECK:
(14) t6 = currentPos == targetPos
```

```
(15) positionReached = t6
```

Each motor control expression is broken into atomic three-address instructions. Safety interlocks at lines 3–7 must evaluate to true before any motion command is emitted, ensuring that the robot arm never moves into an unsafe region.

Task (ii) — Procedure Calls Improving Modularity in Industrial Automation

In a robot automation compiler, procedure calls translate the high-level call statement into a standardised activation record push, parameter transfer, control transfer, and activation record pop sequence. This mechanism provides modularity in the following ways.

Consider three reusable robot control procedures:

```
procedure MoveArm(targetAngle, speed)
    /* move robotic arm to targetAngle at speed */
end procedure

procedure CheckSafety(pos, spd) : bool
    return (pos >= 0 AND pos <= 360 AND spd <= 100)
end procedure

procedure LogEvent(eventCode)
    /* write eventCode to audit log */
end procedure
```

Generated TAC for a call to MoveArm(180, 75):

```
(1)  param 180          ;; push first argument
(2)  param 75           ;; push second argument
(3)  call MoveArm, 2    ;; call with 2 args; return addr pushed
    automatically
(4)  t1 = returnval     ;; capture return value if any
```

Each procedure compiles independently. The compiler generates a single calling sequence for MoveArm regardless of how many times it is called in the program. If the robot arm hardware changes its motion API, only the body of MoveArm needs to be recompiled; all call sites remain unchanged. This modularity also simplifies testing: each procedure can be unit-tested in isolation with simulated sensor inputs before being integrated into the full robot control loop.

Task (iii) — Role of Compiler Optimisation in Manufacturing Efficiency

Real-time manufacturing systems have strict timing requirements. A robotic arm performing assembly must complete each motion command within a defined time window; missing the window causes synchronisation failures between robotic units and may trigger safety shutdowns. Compiler optimisation directly affects the execution time of the compiled robot control software.

Dead code elimination removes unreachable motion commands from the compiled output, reducing the instruction count executed at runtime. Constant propagation replaces variable references with known constant values, for example substituting the literal 75 for motorSpeed throughout the code after the assignment `motorSpeed = 75`, eliminating memory reads and allowing the CPU to schedule instructions more efficiently. Loop unrolling, where the compiler replicates the body of a polling loop multiple times, reduces the overhead of loop control instructions in sensor-polling routines that execute thousands of times per second. Register allocation assigns frequently used variables such as `currentPos` and `motorSpeed` to processor registers rather than memory, eliminating load and store instructions on every access. Together these optimisations reduce the latency of robot control decisions from software execution alone, enabling more rapid response to sensor inputs and improving the throughput and precision of the assembly line.

SET 3

Scenario: Autonomous Drone Delivery System

A logistics company's drone controller compiler processes declarations, assignment statements, and Boolean expressions for navigation, obstacle detection, and battery monitoring. Inefficient intermediate code caused delays in route adjustment and emergency landing.

Task (i) — Intermediate Code for Drone Navigation and Obstacle Detection

Source-level drone control logic:

```
/* Declarations */
float latitude, longitude, altitude, batteryLevel;
bool  obstacleDetected, emergencyLand, routeAdjusted;
int   obstacleDistance, minSafeDistance;

/* Assignments */
minSafeDistance = 5;          /* metres */
batteryLevel    = 78.5;      /* percent */
obstacleDistance = sonarRead();

/* Obstacle detection Boolean */
obstacleDetected = (obstacleDistance < minSafeDistance);

/* Emergency landing condition */
emergencyLand = (batteryLevel < 20.0) OR obstacleDetected;

/* Route adjustment */
if (obstacleDetected AND NOT emergencyLand)
    routeAdjusted = recalculateRoute(latitude, longitude,
    altitude);
else if (emergencyLand)
    initiateEmergencyLanding(latitude, longitude);
```

Three-Address Code (TAC):

```
(1)  minSafeDistance = 5
(2)  batteryLevel    = 78.5
(3)  param 0          ;; no args to sonarRead
(4)  call sonarRead, 0
(5)  obstacleDistance = returnval
(6)  t1 = obstacleDistance < minSafeDistance
(7)  obstacleDetected = t1
(8)  t2 = batteryLevel < 20.0
(9)  t3 = t2 OR obstacleDetected
(10) emergencyLand = t3
```



```
(11) t4 = NOT emergencyLand
(12) t5 = obstacleDetected AND t4
(13) if t5 == false goto L_CHKEMG
(14) param latitude
(15) param longitude
(16) param altitude
(17) call recalculateRoute, 3
(18) routeAdjusted = returnval
(19) goto L_END
(20) L_CHKEMG:
(21) if emergencyLand == false goto L_END
(22) param latitude
(23) param longitude
(24) call initiateEmergencyLanding, 2
(25) L_END:
```

The compiler ensures that all sensor reads (sonarRead) are issued before any Boolean evaluation that depends on them. Emergency landing takes priority because it is checked before route adjustment, reflecting the real-world safety priority ordering of the drone's operating logic.

Task (ii) — Boolean Expressions Supporting Autonomous Decision-Making in Drones

Autonomous drones make real-time decisions without human intervention. Every decision, whether to avoid an obstacle, return to base, or continue on the current route, is encoded as a Boolean expression in the drone's control software. The compiler's correct translation of these Boolean expressions directly determines whether the drone behaves safely.

Compound Boolean expressions such as (batteryLevel < 20.0) OR obstacleDetected capture the disjunction of multiple independent hazard conditions. The compiler applies short-circuit evaluation: if batteryLevel < 20.0 evaluates to true, the entire OR expression is true without evaluating obstacleDetected, saving one sensor read per control cycle. Over thousands of control cycles per flight, this reduction in sensor reads lowers power consumption and reduces the processing latency of the control loop, extending flight time and improving obstacle response speed.

Nested Boolean expressions such as obstacleDetected AND NOT emergencyLand encode priority ordering directly: the drone should only attempt route recalculation if an obstacle is detected but an emergency landing is not already required. The compiler translates these precedence rules faithfully using operator precedence in the parsing phase, ensuring the generated TAC reflects the programmer's intended evaluation order and that safety-critical decisions are never incorrectly short-circuited.

Task (iii) — Importance of Compiler Optimisation in Real-Time Delivery Systems

A drone delivery system operates under hard real-time constraints: obstacle detection must produce an avoidance response within tens of milliseconds, and battery monitoring must update route planning continuously. Compiler optimisation reduces the execution time of these time-critical control loops.

Constant folding evaluates expressions with constant operands at compile time. For example, the expression `minSafeDistance * 2` where `minSafeDistance` is always 5 is replaced by the constant 10 in the compiled code, saving a multiplication at runtime. Dead code elimination removes sensor processing paths that can never be reached given the compiler's analysis of reachable states, reducing the instruction count in the inner control loop. Instruction scheduling reorders the TAC instructions to hide memory latency: sonar read instructions are issued several cycles before their results are needed, allowing the processor to continue with other computation while the sensor data arrives. Register allocation keeps the most frequently referenced navigation variables such as latitude, longitude, and altitude in processor registers throughout the control loop, eliminating repeated memory accesses that would otherwise dominate the loop execution time. Together these optimisations bring the control loop latency within the drone's real-time budget, making reliable autonomous delivery feasible.

SET 4

Scenario: Smart Home Automation System

A home automation company's embedded compiler processes declarations, assignment statements, and Boolean expressions to activate lighting, temperature control, and security devices based on sensor inputs. Inefficient intermediate code generation caused delayed device response and security alert generation.

Task (i) — Intermediate Code for Smart Device Control and Sensor Monitoring

Source-level home automation control logic:

```
/* Declarations */
int  roomTemp, setTemp, lightLevel, motionCount;
bool lightsOn, heaterOn, alarmTriggered, doorLocked;

/* Sensor assignments */
roomTemp  = thermostatRead();
lightLevel = luxSensorRead();
motionCount = motionSensorRead();

/* Temperature control */
heaterOn = (roomTemp < setTemp);

/* Lighting control */
lightsOn = (lightLevel < 300) AND (motionCount > 0);

/* Security alert */
alarmTriggered = (motionCount > 5) AND doorLocked;

/* Execute device actions */
if (heaterOn)
    activateHeater(setTemp);
if (lightsOn)
    setLightBrightness(255);
if (alarmTriggered)
    triggerAlarm("INTRUSION");
```

Three-Address Code (TAC):

```
(1)  call thermostatRead, 0
(2)  roomTemp = returnval
(3)  call luxSensorRead, 0
(4)  lightLevel = returnval
(5)  call motionSensorRead, 0
(6)  motionCount = returnval
```

```
(7)  t1 = roomTemp < setTemp
(8)  heaterOn = t1
(9)  t2 = lightLevel < 300
(10) t3 = motionCount > 0
(11) t4 = t2 AND t3
(12) lightsOn = t4
(13) t5 = motionCount > 5
(14) t6 = t5 AND doorLocked
(15) alarmTriggered = t6
(16) if heaterOn == false goto L_LIGHT
(17) param setTemp
(18) call activateHeater, 1
(19) L_LIGHT:
(20) if lightsOn == false goto L_ALARM
(21) param 255
(22) call setLightBrightness, 1
(23) L_ALARM:
(24) if alarmTriggered == false goto L_END
(25) param "INTRUSION"
(26) call triggerAlarm, 1
(27) L_END:
```

All three sensor reads are issued first (lines 1–6) to allow pipeline overlap. Device control actions are then decided from Boolean evaluations (lines 7–15) and dispatched as conditional procedure calls (lines 16–26). This structure ensures maximum device response speed while maintaining clear, maintainable intermediate code.

Task (ii) — Boolean Expressions Supporting Automated Decision-Making in Smart Homes

Smart home automation relies on continuous evaluation of compound Boolean conditions derived from multiple sensor inputs. The compiler's correct and efficient translation of these conditions directly determines the responsiveness and safety of the home environment.

The condition `lightsOn = (lightLevel < 300) AND (motionCount > 0)` demonstrates multi-sensor fusion in a single Boolean expression. The compiler applies short-circuit evaluation: if `lightLevel >= 300` (the room is already bright enough), the motion sensor check is skipped, saving a sensor read and reducing the control loop latency. This is particularly important in a smart home where tens of sensors may be polled in each control cycle.

The security alarm condition `alarmTriggered = (motionCount > 5) AND doorLocked` encodes a deliberate design decision: the alarm fires only when motion is excessive AND the door is locked, distinguishing an intruder scenario from a family member moving normally around the house. The AND operator's short-circuit semantics mean that if `motionCount <= 5`, the `doorLocked` state

is never read, reducing unnecessary state queries. The compiler must translate this faithfully so that the home automation system does not generate false alarms, which would erode user trust in the system.

Task (iii) — Role of Compiler Optimisation in Home Automation Systems

A smart home controller typically runs on a low-power microcontroller with limited computational resources. Compiler optimisation reduces the resource consumption of the control software without changing its functional behaviour.

Common subexpression elimination identifies sub-expressions that are computed more than once in the control loop and replaces repeated computations with a single stored result. If `motionCount` appears in both the lighting condition and the security condition, the compiler computes it once and reuses the result, saving one sensor read per control cycle. Dead code elimination removes control paths that can never be reached under the current hardware configuration, for example removing heater control code from a room that has no heater, reducing both code size and execution time. Peephole optimisation, applied to the final generated machine code, replaces short sequences of inefficient instructions with faster equivalents. For an embedded microcontroller this can mean the difference between completing the control loop within the required time budget and missing it, which would cause jerky or delayed device response. These optimisations collectively allow the home automation system to run on lower-cost hardware with a smaller battery, reducing the product's bill of materials and extending the device's operational lifetime between charges.

SET 5

Scenario: Smart Parking Management System

A smart city company's parking management compiler processes declarations, assignment statements, Boolean expressions, and case statements to manage slot allocation and fee calculation. Inefficient intermediate code caused delays in slot assignment and payment confirmation during peak hours.

Task (i) — Intermediate Code for Parking Slot Allocation and Fee Calculation

Source-level parking management logic:

```
/* Declarations */
int  totalSlots, availableSlots, vehicleType, slotID;
float parkingDuration, feePerHour, totalFee;
bool  slotAvailable, paymentConfirmed;

/* Sensor / input assignments */
totalSlots    = 200;
availableSlots = readSensor("SLOT_COUNT");
vehicleType   = readInput("VEHICLE_TYPE"); /* 1=car, 2=bike,
3=truck */
parkingDuration = readInput("DURATION");    /* hours */

/* Slot availability check */
slotAvailable = (availableSlots > 0);

/* Fee calculation using case */
case vehicleType of
    1: feePerHour = 30.0;    /* car */
    2: feePerHour = 15.0;    /* bike */
    3: feePerHour = 60.0;    /* truck */
    default: feePerHour = 30.0;
end case

/* Total fee */
if (slotAvailable)
    slotID = allocateSlot();
    totalFee = parkingDuration * feePerHour;
    paymentConfirmed = processPayment(totalFee);
```

Three-Address Code (TAC):

- ```
(1) totalSlots = 200
(2) param "SLOT_COUNT"
```

```

(3) call readSensor, 1
(4) availableSlots = returnval
(5) param "VEHICLE_TYPE"
(6) call readInput, 1
(7) vehicleType = returnval
(8) param "DURATION"
(9) call readInput, 1
(10) parkingDuration = returnval
(11) t1 = availableSlots > 0
(12) slotAvailable = t1

;; Case statement – jump table for vehicleType
(13) if vehicleType == 1 goto L_CAR
(14) if vehicleType == 2 goto L_BIKE
(15) if vehicleType == 3 goto L_TRUCK
(16) goto L_DEFAULT
(17) L_CAR: feePerHour = 30.0 ; goto L_CASE_END
(18) L_BIKE: feePerHour = 15.0 ; goto L_CASE_END
(19) L_TRUCK: feePerHour = 60.0 ; goto L_CASE_END
(20) L_DEFAULT: feePerHour = 30.0
(21) L_CASE_END:

;; Slot allocation and payment
(22) if slotAvailable == false goto L_FULL
(23) call allocateSlot, 0
(24) slotID = returnval
(25) t2 = parkingDuration * feePerHour
(26) totalFee = t2
(27) param totalFee
(28) call processPayment, 1
(29) paymentConfirmed = returnval
(30) goto L_END
(31) L_FULL: /* no slots available – notify user */
(32) L_END:

```

The case statement is translated into a chain of conditional jumps (lines 13–20), one per vehicle type. For a larger number of vehicle types, the compiler would generate a jump table for  $O(1)$  dispatch instead of  $O(n)$  sequential comparison, further improving performance during peak hours.

### **Task (ii) — Boolean Expressions and Case Statements Managing Parking Operations**

The smart parking system uses Boolean expressions and case statements together to implement two distinct kinds of control logic.

Boolean expressions handle binary decisions with continuous sensor data. The condition `slotAvailable = (availableSlots > 0)` encodes the go/no-go decision for slot allocation. With short-circuit evaluation, if `availableSlots` is zero the compiler immediately jumps to the "full" handler without evaluating any further conditions, minimising the time before the user receives a "no slots" notification. This rapid response is essential during peak hours when hundreds of vehicles may be querying the system simultaneously.

Case statements handle categorical decisions where the input belongs to one of a finite, enumerated set of categories. Translating `vehicleType` into a fee rate is exactly this kind of decision. The case statement's intermediate code uses a jump table, which dispatches to the correct fee assignment in constant time regardless of how many vehicle categories exist. This is superior to a chain of if-else statements, which has linear dispatch time growing with the number of categories. The compiler further optimises the case table by sorting entries and using binary search for large category sets, balancing code size against dispatch speed. For the parking system, this ensures that fee calculation for every vehicle type takes a predictable, bounded time, supporting consistent transaction throughput under peak load.

### **Task (iii) — Role of Compiler Optimisation in Smart Parking Management Systems**

A smart parking system processes potentially thousands of simultaneous slot queries, payment transactions, and sensor updates during peak hours. Compiler optimisation of the control software directly affects the system's ability to sustain high transaction throughput without delays.

Inlining small frequently-called procedures such as `readSensor` eliminates the procedure call overhead, which includes pushing parameters, saving the return address, jumping to the procedure, executing the return jump, and restoring registers. For a sensor read that consists of a single memory-mapped register access, the overhead of a full procedure call can be larger than the sensor read itself. Inlining replaces the call with the sensor read instruction directly, reducing the instruction count per control cycle. Loop optimisations such as loop-invariant code motion move computations whose results do not change within a loop, for example the `feePerHour` assignment, to outside the loop body, so the computation executes once rather than on every iteration. For the parking system's main control loop that polls all entry sensors every few milliseconds, this can reduce the per-iteration instruction count substantially. Strength reduction replaces expensive operations with cheaper equivalents: multiplying `parkingDuration` by an integer `feePerHour` can be replaced by repeated addition when `feePerHour` is small, saving a multiplication instruction on microcontrollers that implement multiplication in software. Together these optimisations allow the parking management system to handle peak load on



## RUBRICS FOR CALCULATION

| Numerical Problems        |           |                                                                                   |                                           |                                            |                                               |
|---------------------------|-----------|-----------------------------------------------------------------------------------|-------------------------------------------|--------------------------------------------|-----------------------------------------------|
| Criteria                  | Weightage | Level 4<br>(Exemplary)                                                            | Level 3<br>(Proficient)                   | Level 2<br>(Developing)                    | Level 1<br>(Beginning)                        |
| Problem Understanding     | 20%       | Clearly interprets problem, identifies all parameters and requirements accurately | Understands problem with minor gaps       | Partial understanding; misses key elements | Misinterprets or unable to understand problem |
| Method Selection          | 20%       | Selects most appropriate method/formula with strong justification                 | Selects correct method with minor errors  | Inappropriate or partially correct method  | Unable to select method                       |
| Solution Process          | 25%       | Logical, systematic steps with correct approach throughout                        | Mostly correct steps with minor mistakes  | Disorganized steps; multiple errors        | No clear process                              |
| Accuracy of Calculation   | 20%       | All calculations accurate; correct final answer                                   | Minor calculation errors                  | Frequent errors; incorrect final answer    | No valid calculations                         |
| Interpretation of Results | 15%       | Clearly interprets results with units and engineering relevance                   | Basic interpretation with minor omissions | Limited or unclear interpretation          | No interpretation                             |